System Analysis

DR. DRAGUNOV



System development methodologies

Originally Agile manifesto was signed by:

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

Agile Development

Agile manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- ***Individuals and interactions*** *over processes and tools*
- ***Working software*** *over comprehensive documentation*
- ***Customer collaboration*** *over contract negotiation*
- ***Responding to change*** *over following a plan*

That is, while there is value in the items on the right, we value the items on the left more.



The Twelve Principle of Agile Manifesto

Customer Satisfaction: Manifesto provides high priority to satisfy the costumer's requirements. This is done through early and continuous delivery of valuable software.

Welcome Change: Making changes during software development is common and inevitable. Every changing requirement should be welcome, even in the late development phase. Agile process works to increase the customers' competitive advantage.

Deliver the Working Software: Deliver the working software frequently, ranging from a few weeks to a few months with considering the shortest time period.

Collaboration: Business people (Scrum Master and Project Owner) and developers must work together during the entire life of a project development phase.

Motivation: Projects should be build around motivated team members. Provide such environment that supports individual team members and trust them. It makes them feel responsible for getting the job done thoroughly.

Face-to-face Conversation: Face-to-face conversation between Scrum Master and development team and between the Scrum Master and customers for the most efficient and effective method of conveying information to and within a development team.

The Twelve Principle of Agile Manifesto

Measure the Progress as per the Working Software: The working software is the key and primary measure of the progress.

Maintain Constant Pace: The aim of agile development is sustainable development. All the businesses and users should be able to maintain a constant pace with the project.

Monitoring: Pay regular attention to technical excellence and good design to maximize agility.

Simplicity: Keep things simple and use simple terms to measure the work that is not completed.

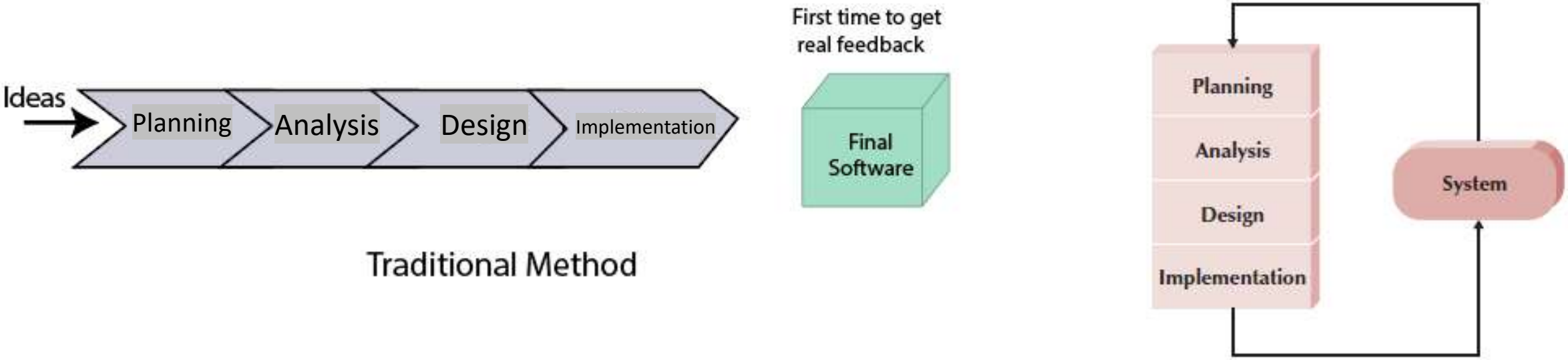
Self-organized Teams: The Agile team should be self-organized. They should not be depending heavily on other teams because the best architectures, requirements, and designs emerge from self-organized teams.

Review the Work Regularly: The work should be reviewed at regular intervals, so that the team can reflect on how to become more productive and adjust its behavior accordingly.

The Cargo Cults believe that manufactured western goods ('cargo') have been created by ancestral spirits and intended for Melanesian people.



Typical Agile Development Methodology



The Scrum

Scrum creators: Jeff Sutherland and Ken Schwaber



Scrum has been used by:

- Microsoft
- Yahoo
- Google
- Electronic Arts
- IBM
- Lockheed Martin
- Apple

Etc.



Scrum has been used for:

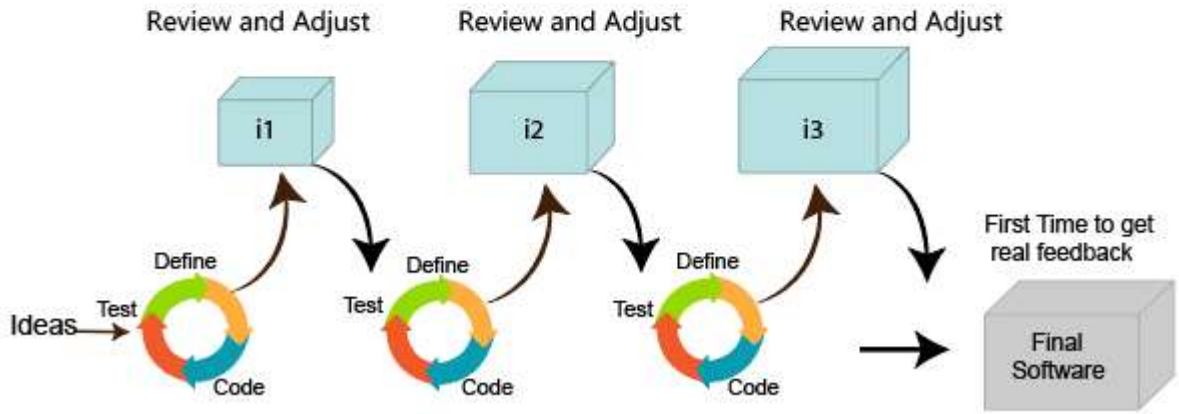
- Commercial software
- In-house development
- Contract development
- Fixed-price projects
- Financial applications
- ISO 9001-certified applications
- Embedded systems
- 24x7 systems with 99.999% uptime requirements
- the Joint Strike Fighter
- Video game development
- FDA-approved, life-critical systems
- Satellite-control software
- Websites
- Handheld software
- Mobile phones
- Network switching applications
- ISV applications
- Some of the largest applications in use

What is Agile “Methodology” or Philosophy?

An agile methodology is an iterative approach to software development. Each iteration of agile methodology takes a short time interval of 1 to 4 weeks. The agile development process is aligned to deliver the changing business requirement. It distributes the software with faster and fewer changes.

The single-phase software development takes 6 to 18 months. In single-phase development, all the requirement gathering and risks management factors are predicted initially.

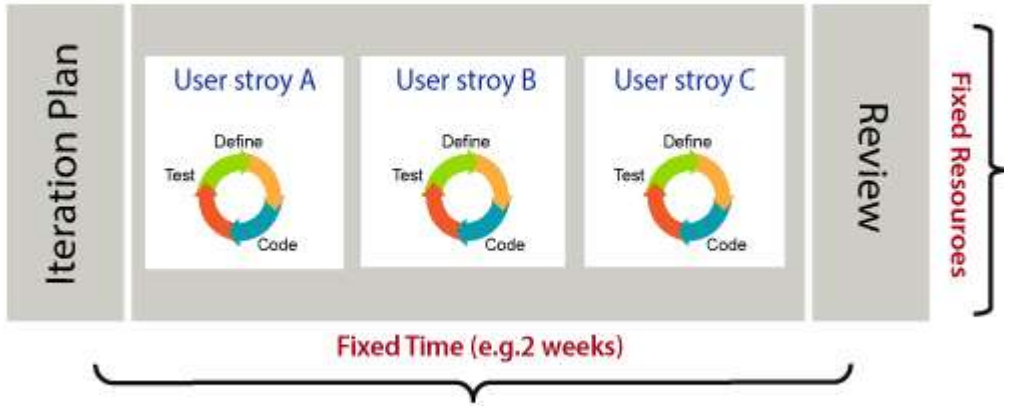
The agile software development process frequently takes the feedback of workable product. The workable product is delivered within 1 to 4 weeks of iteration.



Agile Method

An Agile methodology is not a specific set of ceremonies or specific development techniques. Rather, it is a group of methodologies that demonstrate a commitment to tight feedback cycles and continuous improvement. An Agile team works in iterations to deliver the customer requirement, and each iteration takes 10 to 15 days. However, the original Agile Manifesto didn't set the time period of two-week iterations or an ideal team size.

Each user requirement is a planned based and their backlog prioritization and size. The team decides, how much scope they have and how many hours available with each team to perform their planed task.

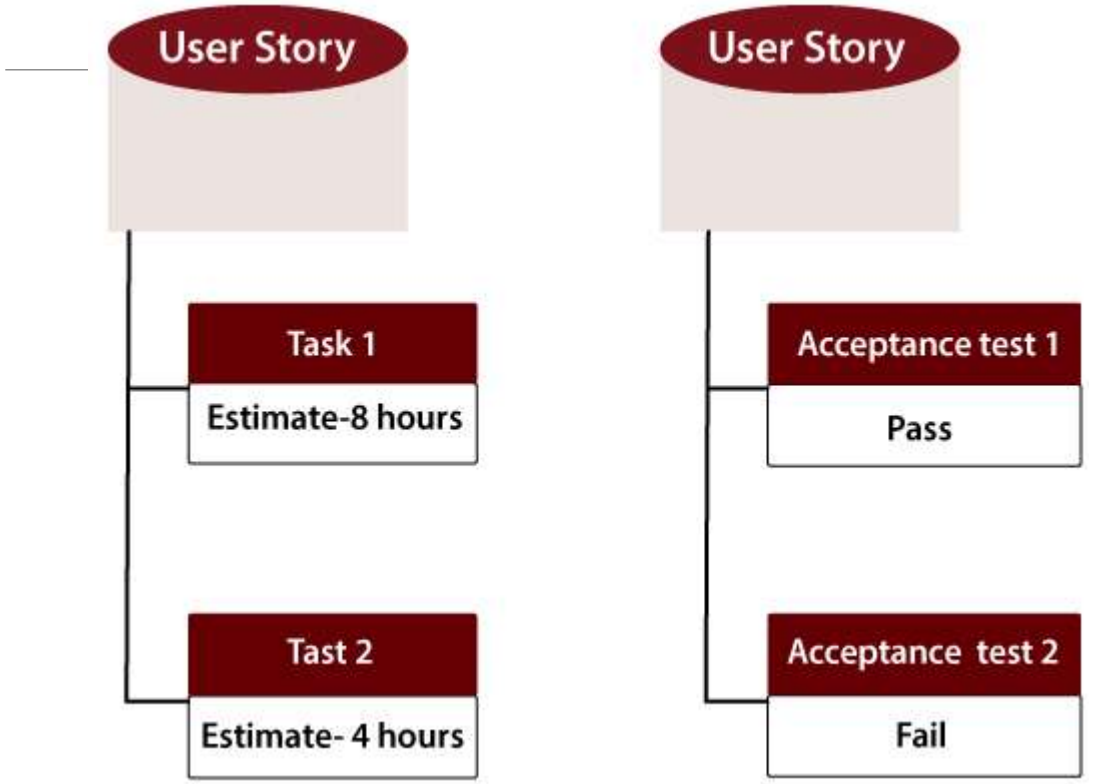


What is a user requirement?

The user requirement defines the requirements of the user in terms of functionalities. There may be of two type of functionality.

- As a <User Role> I want <Functionality> so that <Business Value>
- In order to <Business value> as a <User Role> I want <Functionality>.

During software release planning, a rough estimate is given to a user requirement using relative scale points. During iteration planning, the requirement is broken down into tasks.



Relation between User requirement and Task

- User requirement talks about what is to be done. It defines the needs of users.
- Task talks about how it is to be done. It defines how functionality is implemented.
- User requirements are implemented by tasks. Every requirement is gathering as the task.
- User requirement is divided into different tasks when it is planned in current iteration.
- User tasks are estimated in hours based, generally it is between 2 to 12 hours.
- Requirements are validated using acceptance test.

Comparison between the Agile methodology and Waterfall model:

Agile methodology	Waterfall model
It follows the incremental approach.	It is a sequential design process.
It divides the project development lifecycle into a sprint.	The software development process is divided into distinct phases.
Agile methodology is a flexible methodology.	The Waterfall is a structured software development methodology.
Agile is the collection of many different projects.	It is completed as one single project.
The test plan is reviewed after each sprint	Test plan is reviewed after complete development.
Testing team can take part in the requirements change phase without problems.	It is difficult for the test to initiate any change in needs.

Definitions of Scrum

Scrum: A framework within which people can address complex adaptive problems, while productively and creatively delivering products of the highest possible value.

Scrum is an agile project management framework that helps teams structure and manage their work through a set of values, principles, and practices.

Scrum is not a process, technique, or definitive method. Rather, **it is a framework within which you can employ various processes and techniques.**

The Scrum framework consists of <u>Scrum Team members and their associated roles, events, artifacts, and rules.</u>

Each component within the framework serves a specific purpose and is essential to Scrum's success and usage.

The rules of Scrum bind together the roles, events, and artifacts, governing the relationships and interaction between them.

The Scrum Team

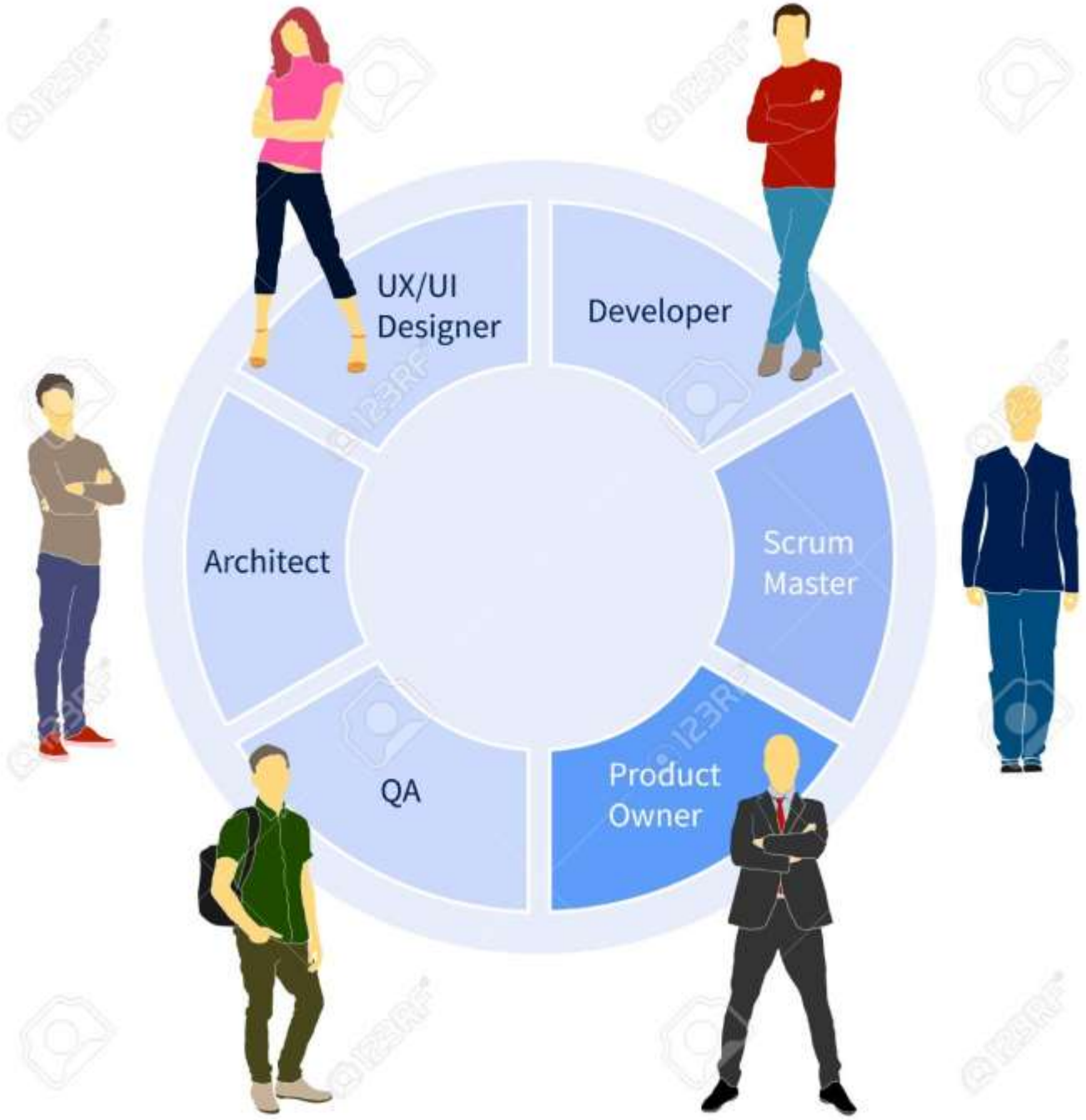
The Scrum Team consists of a **Product Owner**, the **Development Team**, and a **Scrum Master**.

Scrum Teams are self-organizing and cross-functional.

Self-organizing teams choose how best to accomplish their work, rather than being directed by others outside the team.

Cross-functional teams have all competencies needed to accomplish the work without depending on others not part of the team.

The team model in Scrum is designed to optimize flexibility, creativity, and productivity.



The Product Owner



The Product Owner is responsible for maximizing the value of the product resulting from work of the Development Team. How this is done may vary widely across organizations, Scrum Teams, and individuals.

- Define the features of the product
- Makes scope vs. schedule decisions
- Responsible for achieving financial goals of the project
- Prioritize the product backlog
- Adjust features and priority every sprint, as needed
- Accept or reject work results

The Development Team

The Development Team consists of professionals who do the work of delivering a potentially releasable Increment of “Done” product at the end of each Sprint.

A “Done” increment is required at the Sprint Review.

Only members of the Development Team create the Increment.

Development Teams are structured and empowered by the organization to organize and manage their own work.

The resulting synergy optimizes the Development Team’s overall efficiency and effectiveness.



Development Teams characteristics:

- **They are self-organizing**. No one (not even the Scrum Master) tells the Development Team how to turn Product Backlog into Increments of potentially releasable functionality;
- **Development Teams are cross-functional**, with all the skills as a team necessary to create a product Increment;
- **Scrum recognizes no titles for Development Team members**, regardless of the work being performed by the person;
- **Scrum recognizes no sub-teams in the Development Team**, regardless of domains that need to be addressed like testing, architecture, operations, or business analysis; and,
- Individual Development Team members may have specialized skills and areas of focus, but accountability belongs to the Development Team as a whole.

Development Team Size

Optimal Development Team size is small enough to remain nimble and large enough to complete significant work within a Sprint.

Smaller Development Teams may encounter skill constraints during the Sprint, causing the Development Team to be unable to deliver a potentially releasable Increment.

Having more than nine members requires too much coordination. Large Development Teams generate too much complexity for an empirical process to be useful.

The Product Owner and Scrum Master roles are not included in this count unless they are also executing the work of the Sprint Backlog.

- Typically 5-9 people
- Cross-functional:
- Programmers, testers, user experience designers, etc.
- Members should be full-time
- May be exceptions (e.g., database administrator)
- Teams are self-organizing
- Ideally, no titles but rarely a possibility
- Membership should change only between sprints



The Scrum Master

The Scrum Master is responsible for promoting and supporting Scrum as defined in the Scrum Guide.

Scrum Masters do this by helping everyone understand Scrum theory, practices, rules, and values.

The Scrum Master is a servant-leader for the Scrum Team.

The Scrum Master helps those outside the Scrum Team understand which of their interactions with the Scrum Team are helpful and which aren't. The Scrum Master helps everyone change these interactions to maximize the value created by the Scrum Team.



INTERNATIONAL SCRUM INSTITUTE ™
SMAC ™
Scrum Master Accredited Certification ™
Price: USD 49.-
Lifetime Valid in all Countries

INTERNATIONAL SCRUM INSTITUTE ™
SPOAC ™
Scrum Product Owner Accredited Certification ™
Price: USD 69.-
Lifetime Valid in all Countries

Scrum Master Service to the Product Owner

The Scrum Master serves the Product Owner in several ways, including:

- Ensuring that goals, scope, and product domain are understood by everyone on the Scrum Team as well as possible;
- Finding techniques for effective Product Backlog management;
- Helping the Scrum Team understand the need for clear and concise Product Backlog items;
- Understanding product planning in an empirical environment;
- Ensuring the Product Owner knows how to arrange the Product Backlog to maximize value;
- Understanding and practicing agility; and,
- Facilitating Scrum events as requested or needed.

Scrum Master Service to the Development Team

The Scrum Master serves the Development Team in several ways, including:

- Coaching the Development Team in self-organization and cross-functionality;
- Helping the Development Team to create high-value products;
- Removing impediments to the Development Team's progress;
- Facilitating Scrum events as requested or needed; and,
- Coaching the Development Team in organizational environments in which Scrum is not yet fully adopted and understood.



Scrum Master Service to the Organization

The Scrum Master serves the organization in several ways, including:

- Leading and coaching the organization in its Scrum adoption;
- Planning Scrum implementations within the organization;
- Helping employees and stakeholders understand and enact Scrum and empirical product development;
- Causing change that increases the productivity of the Scrum Team; and,
- Working with other Scrum Masters to increase the effectiveness of the application of Scrum in the organization.

Scrum Events

Prescribed events are used in Scrum to create regularity and to minimize the need for meetings not defined in Scrum.

All events are time-boxed events, such that every event has a maximum duration.

Once a **Sprint** begins, its duration is fixed and cannot be shortened or lengthened.

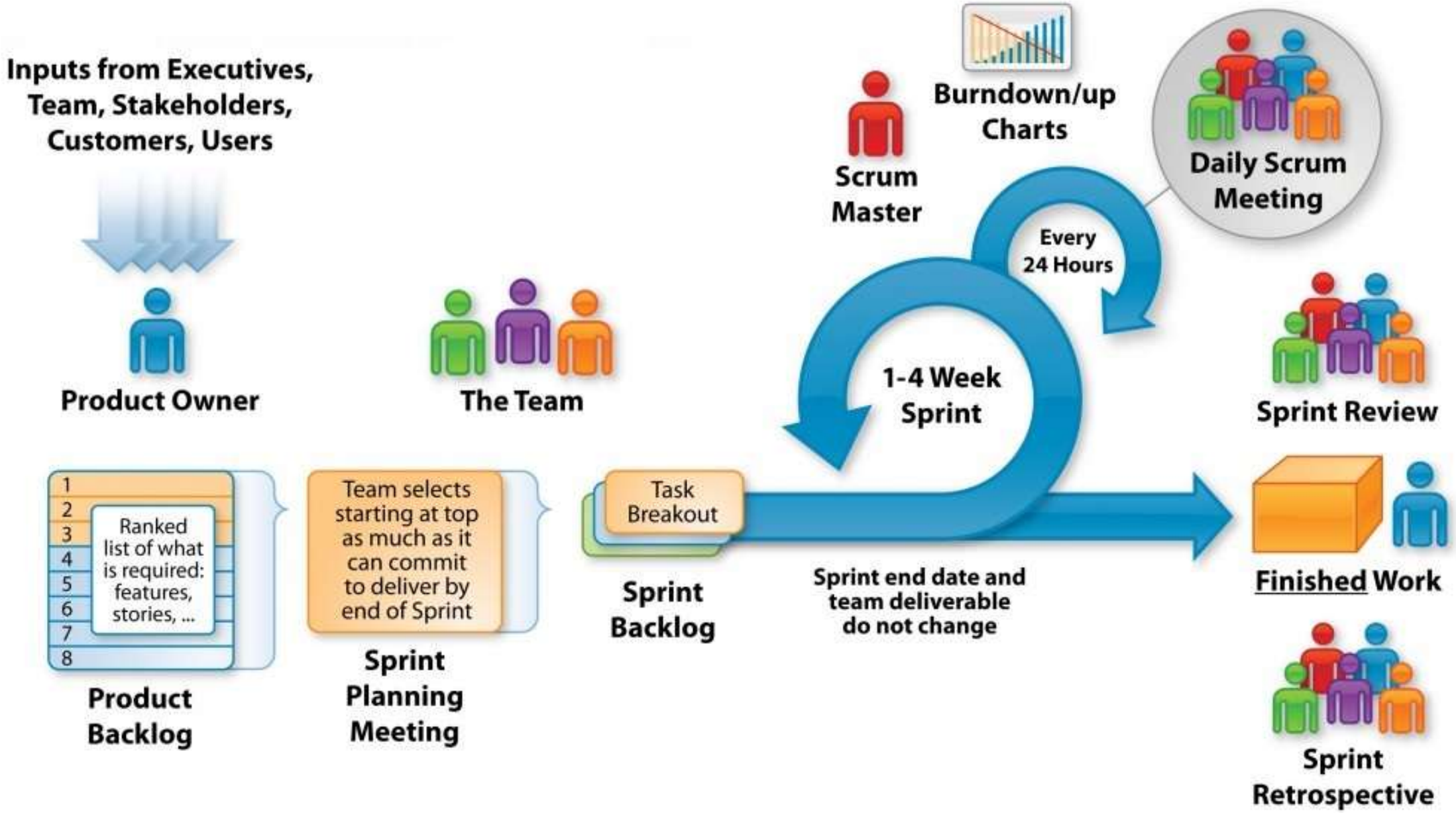
The remaining events may end whenever the purpose of the event is achieved, ensuring an appropriate amount of time is spent without allowing waste in the process.

Other than the Sprint itself, which is a container for all other events, each event in Scrum is a formal opportunity to inspect and adapt something.

These events are specifically designed to enable critical transparency and inspection.

Failure to include any of these events results in reduced transparency and is a lost opportunity to inspect and adapt.

The Sprint


Inputs from Executives, Team, Stakeholders, Customers, Users
Product Owner
The Team
Scrum Master
Burndown/up Charts
Daily Scrum Meeting
Every 24 Hours
1-4 Week Sprint
Sprint Review
1
2
3
4
5
6
7
8
Ranked list of what is required: features, stories, ...
Product Backlog
Team selects starting at top as much as it can commit to deliver by end of Sprint
Sprint Planning Meeting
Task Breakout
Sprint Backlog
Sprint end date and team deliverable do not change
Finished Work
Sprint Retrospective

The heart of Scrum is a Sprint, a time-box of one month or less during which a "Done", useable, and potentially releasable product Increment is created.

Sprints have consistent durations throughout a development effort. A new Sprint starts immediately after the conclusion of the previous Sprint.

Sprints contain and consist of the **Sprint Planning**, **Daily Scrums**, the **development work**, the **Sprint Review**, and the **Sprint Retrospective**.

During the Sprint:

- No changes are made that would endanger the Sprint Goal;
- Quality goals do not decrease; and,
- Scope may be clarified and re-negotiated between the Product Owner and Development Team as more is learned.

Each Sprint may be considered a project with no more than a one-month horizon.

Cancelling a Sprint



A Sprint can be cancelled before the Sprint time-box is over.

Only the Product Owner has the authority to cancel the Sprint, although he or she may do so under influence from the stakeholders, the Development Team, or the Scrum Master.

A Sprint would be cancelled if the Sprint Goal becomes obsolete. This might occur if the company changes direction or if market or technology conditions change. In general, a Sprint should be cancelled if it no longer makes sense given the circumstances. But, due to the short duration of Sprints, cancellation rarely makes sense. When a Sprint is cancelled, any completed and “Done” Product Backlog items are reviewed.

If part of the work is potentially releasable, the Product Owner typically accepts it. All incomplete Product Backlog Items are re-estimated and put back on the Product Backlog. The work done on them depreciates quickly and must be frequently re-estimated. Sprint cancellations consume resources, since everyone regroups in another Sprint Planning to start another Sprint. Sprint cancellations are often traumatic to the Scrum Team, and are very uncommon.

Sprint Planning

The work to be performed in the Sprint is planned at the Sprint Planning.

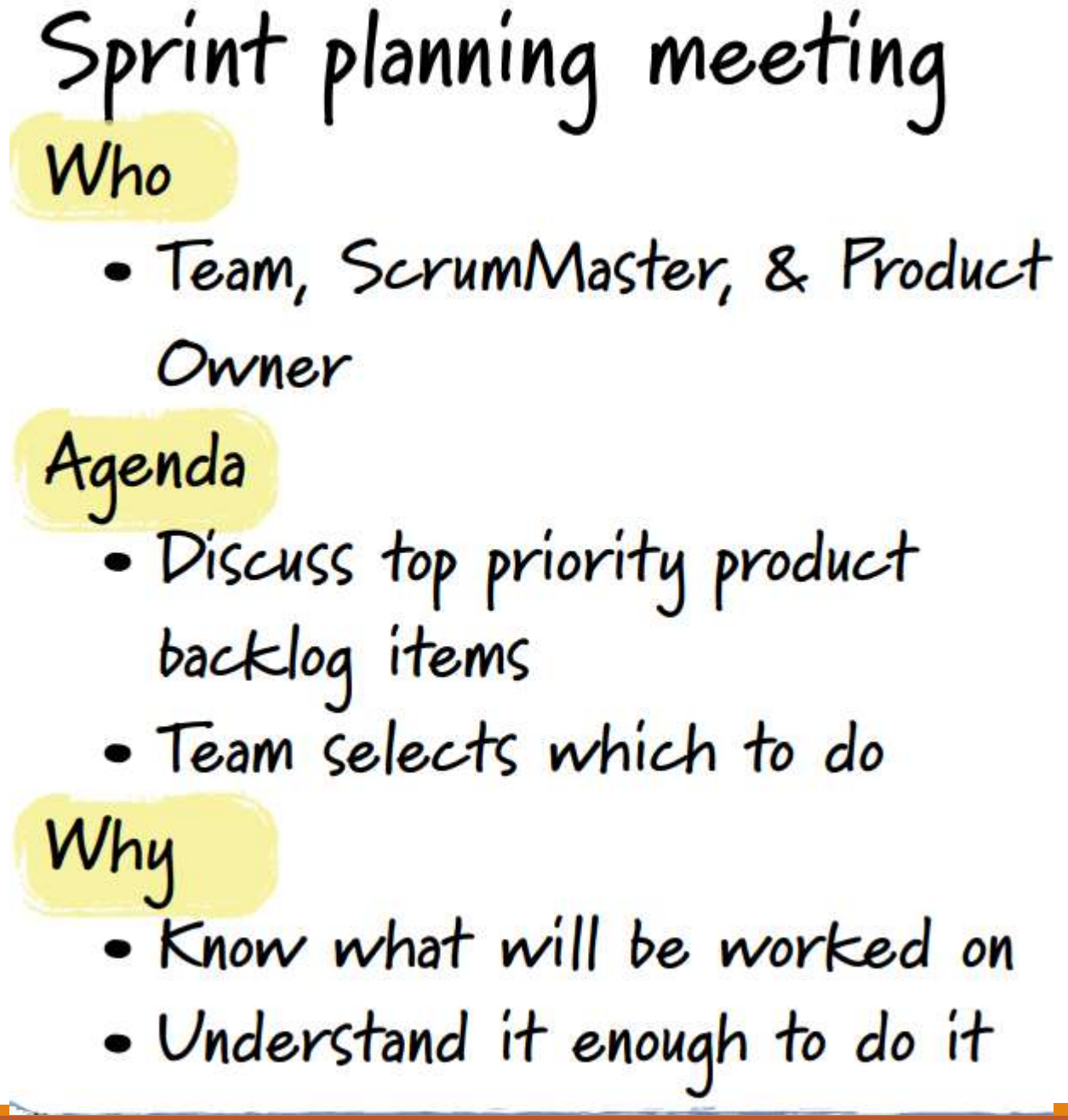
This plan is created by the collaborative work of the entire Scrum Team.

Sprint Planning is time-boxed to a maximum of eight hours for a one-month Sprint.

For shorter Sprints, the event is usually shorter. The Scrum Master ensures that the event takes place and that attendants understand its purpose. The Scrum Master teaches the Scrum Team to keep it within the time-box.

Sprint Planning answers the following:

- What can be delivered in the Increment resulting from the upcoming Sprint?
- How will the work needed to deliver the Increment be achieved?



Sprint Goal

The Sprint Goal is an objective set for the Sprint that can be met through the implementation of Product Backlog.

It provides guidance to the Development Team on why it is building the Increment. It is created during the Sprint Planning meeting.

The Sprint Goal gives the Development Team some flexibility regarding the functionality implemented within the Sprint. The selected Product Backlog items deliver one coherent function, which can be the Sprint Goal.

Factors affecting Sprint planning

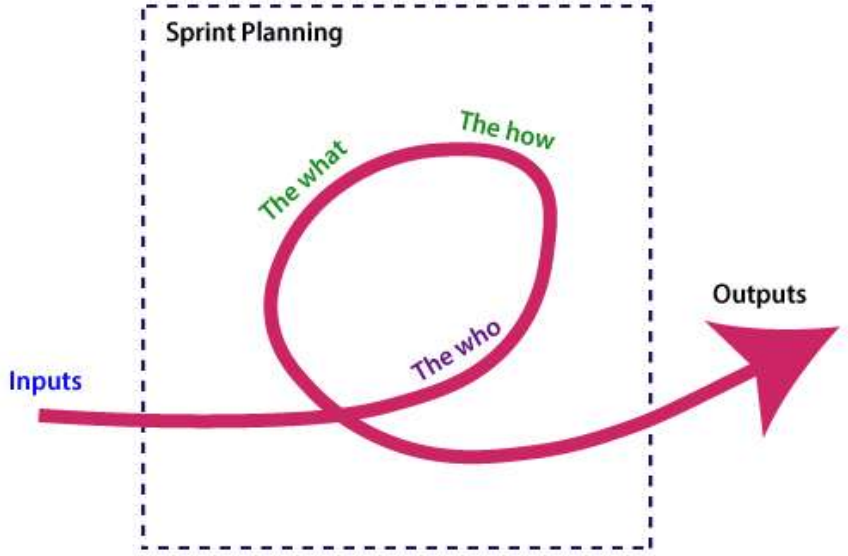
The What: The product owner describes the goal of the sprint and the backlog items which contribute to achieve that goal.

The How: Agile development team plans its necessary work on how to achieve and deliver the sprint goal.

The Who: The product owner defines the goal based on the value that the customers seek. And the developer needs to understand how they can or cannot deliver that goal.

The Inputs: The product backlog provides the list of input stuff that could potentially be part of the current sprint. The team looks over the existing work done in incremental ways.

The Outputs: The critical outcome of sprint planning is to meet described team goal. The product set the goal of sprint and how they will start working towards the goal.



Daily Scrum

The Daily Scrum is a 15-minute time-boxed stand-up event for the Development Team.

The Daily Scrum is held every day of the Sprint. At it, the Development Team plans work for the next 24 hours.

The Daily Scrum is held at the same time and place each day to reduce complexity. The Development Team uses the Daily Scrum to inspect progress toward the Sprint Goal and to inspect how progress is trending toward completing the work in the Sprint Backlog.

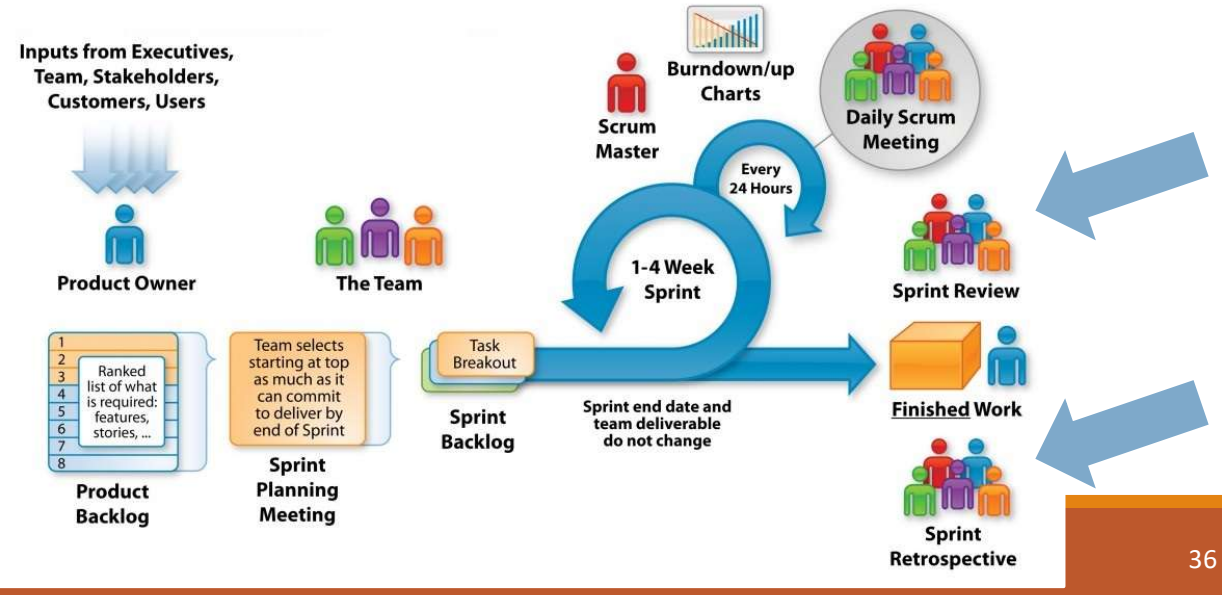


The structure of the meeting is set by the Development Team and can be conducted in different ways if it focuses on progress toward the Sprint Goal. Some Development Teams will use questions, some will be more discussion based. Here is an example of what might be used:



Sprint Review

- Attendees include the Scrum Team and key stakeholders invited by the Product Owner;
- The Product Owner explains what Product Backlog items have been “Done” and what has not been “Done”;
- The Development Team discusses what went well during the Sprint, what problems it ran into, and how those problems were solved;
- The Development Team demonstrates the work that it has “Done” and answers questions about the Increment;



Sprint Review cont…

- The Product Owner discusses the Product Backlog as it stands. He or she projects likely target and delivery dates based on progress to date (if needed);
- The entire group collaborates on what to do next, so that the Sprint Review provides valuable input to subsequent Sprint Planning;
- Review of how the marketplace or potential use of the product might have changed what is the most valuable thing to do next; and,
- Review of the timeline, budget, potential capabilities, and marketplace for the next anticipated releases of functionality or capability of the product.

Sprint Retrospective

The Sprint Retrospective is an opportunity for the Scrum Team to inspect itself and create a plan for improvements to be enacted during the next Sprint.

The Sprint Retrospective occurs after the Sprint Review and prior to the next Sprint Planning.

This is at most a three-hour meeting for one-month Sprints. For shorter Sprints, the event is usually shorter. The Scrum Master ensures that the event takes place and that attendants understand its purpose.

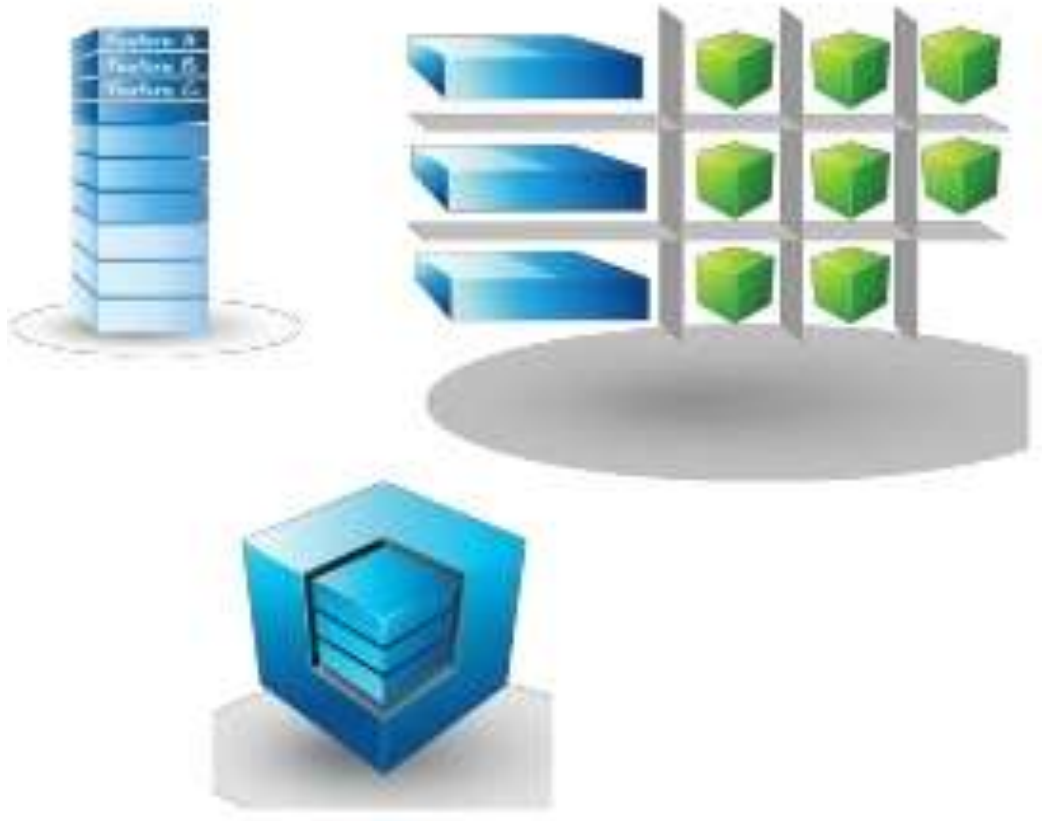
The purpose of the Sprint Retrospective is to:

- Inspect how the last Sprint went with regards to people, relationships, process, and tools;
- Identify and order the major items that went well and potential improvements; and,
- Create a plan for implementing improvements to the way the Scrum Team does its work.

Scrum Artifacts

Scrum's artifacts represent work or value to provide transparency and opportunities for inspection and adaptation. Artifacts defined by Scrum are specifically designed to maximize transparency of key information so that everybody has the same understanding of the artifact

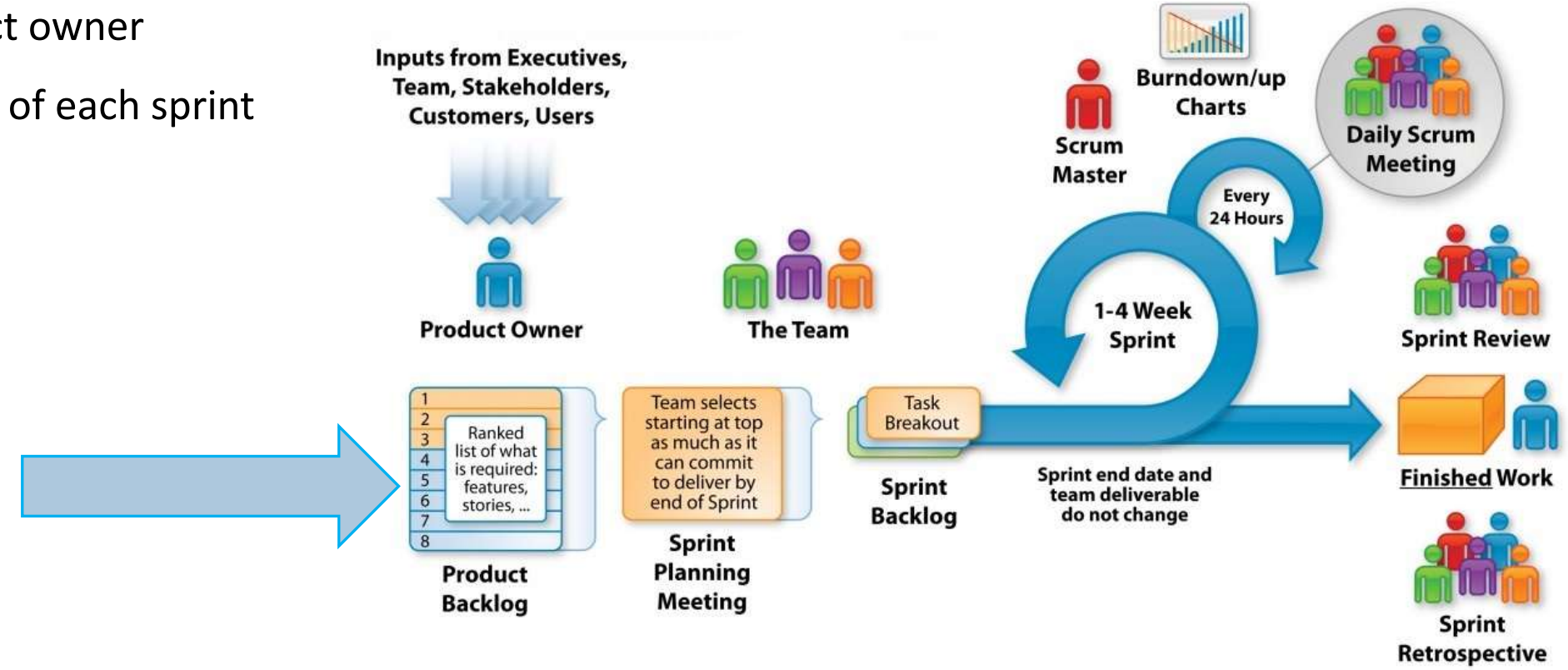
- Product backlog
- Sprint backlog



Product Backlog

The requirements

- A list of all desired work on the project
- Ideally expressed such that each item has value to the users or customers of the product
- Prioritized by the product owner
- Reprioritized at the start of each sprint



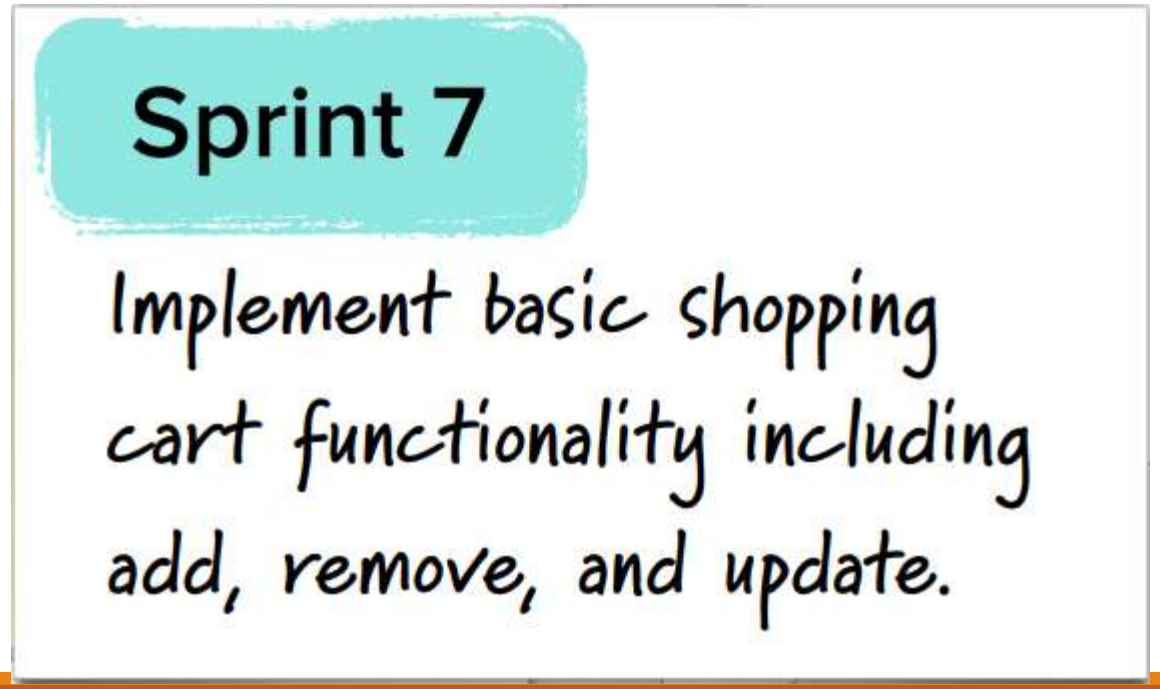
Product Backlog

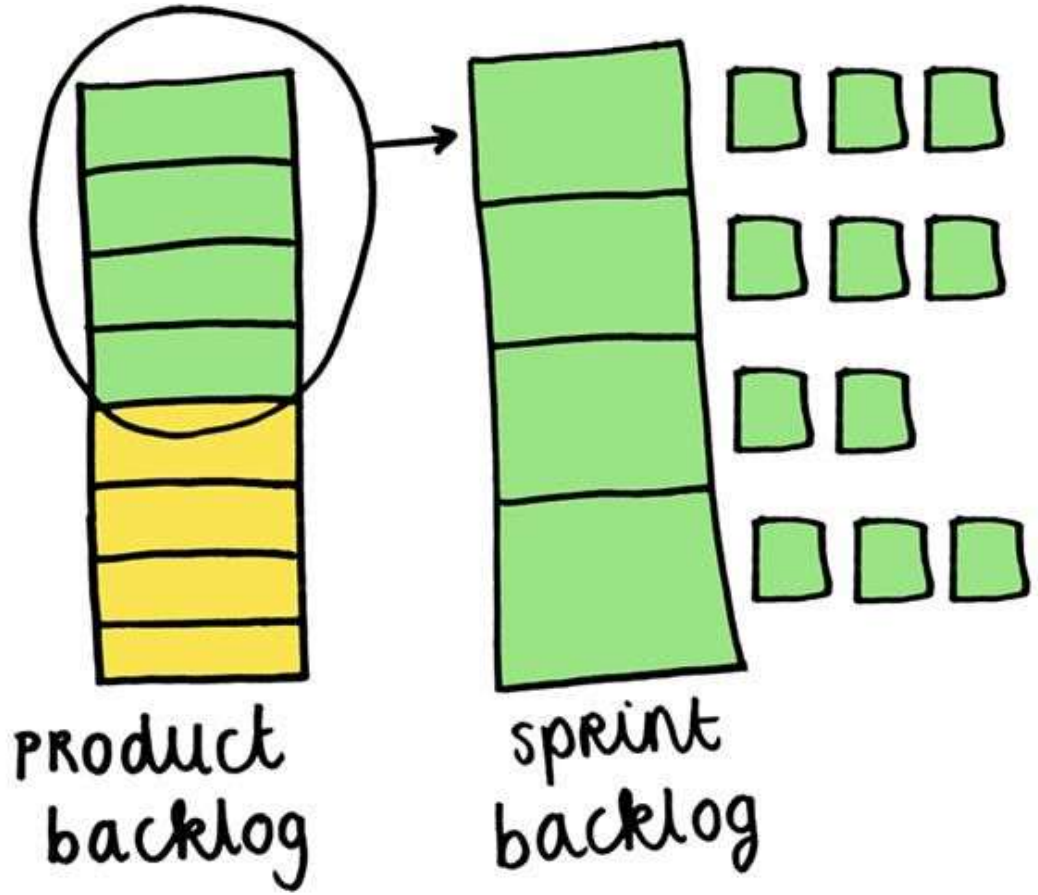
Backlog item	Estimate
Allow a guest to make a reservation	3
As a guest, I want to cancel a reservation.	5
As a guest, I want to change the dates of a reservation.	3
As a hotel employee, I can run RevPAR reports (revenue-per-available-room)	8
Improve exception handling	8
...	30
...	50

Sprint Backlog

The Sprint Backlog is the set of Product Backlog items selected for the Sprint, plus a plan for delivering the product Increment and realizing the Sprint Goal.

The Sprint Backlog is a forecast by the Development Team about what functionality will be in the next Increment and the work needed to deliver that functionality into a “Done” Increment.





What is the product backlog?

A product backlog is a registered list of work for the development team. It is driven from the roadmap and its requirements. The essential task is represented at the top of the product backlog so that the team member knows what to deliver first. The developer team doesn't work through the backlog from the product owner's side and product owner doesn't push the work to the developer team. The developer team pulls work from the product backlog.

Backlog starts with the two "R"s

The fundamental product backlog is provided by a team's **roadmap** and **requirements**. Roadmap repetition breaks down into several epics, and each epic will have several requirements and user stories.

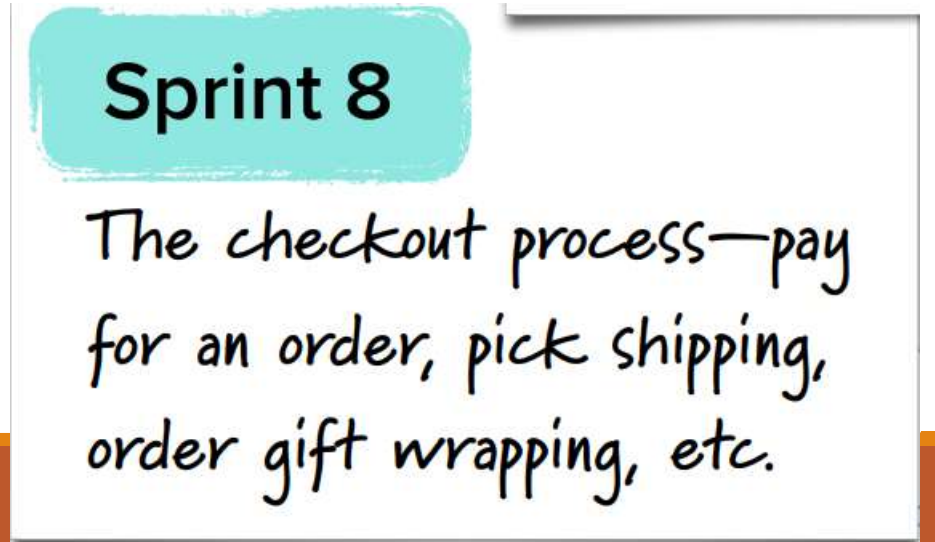
The product owner organizes each of the customer stories into a single list. This story is organized for the development team. The product owner chooses to deliver first complete epic.

The factors that influence a product owner's prioritization

- Priority of customer
- Importance of getting feedback
- Relative implementation difficulty
- Symbiotic relationships between work items

Managing the sprint backlog

- Individuals sign up for work of their own choosing
- Work is never assigned
- Estimated work remaining is updated daily
- Any team member can add, delete or change the sprint backlog
- Work for the sprint emerges
- If work is unclear, define a sprint backlog item with a larger amount of time and break it down later
- Update work remaining as more becomes known



Artifact Transparency

Scrum relies on transparency. Decisions to optimize value and control risk are made based on the perceived state of the artifacts.

The Scrum Master must work with the Product Owner, Development Team, and other involved parties to understand if the artifacts are completely transparent.

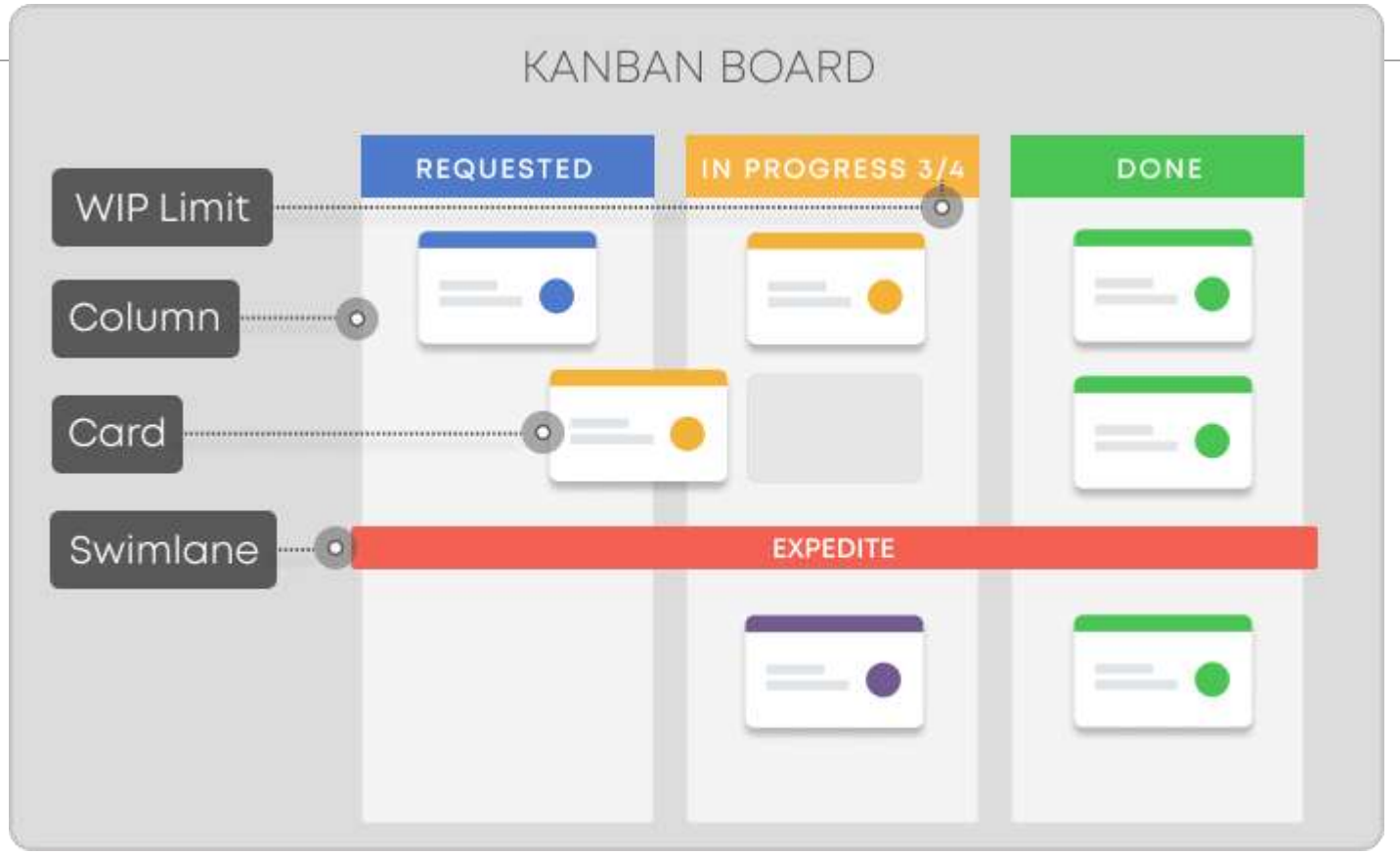
Definition of “Done”

When a Product Backlog item or an Increment is described as “Done”, everyone must understand what “Done” means.

Although this may vary significantly per Scrum Team, members must have a shared understanding of what it means for work to be complete, to ensure transparency.

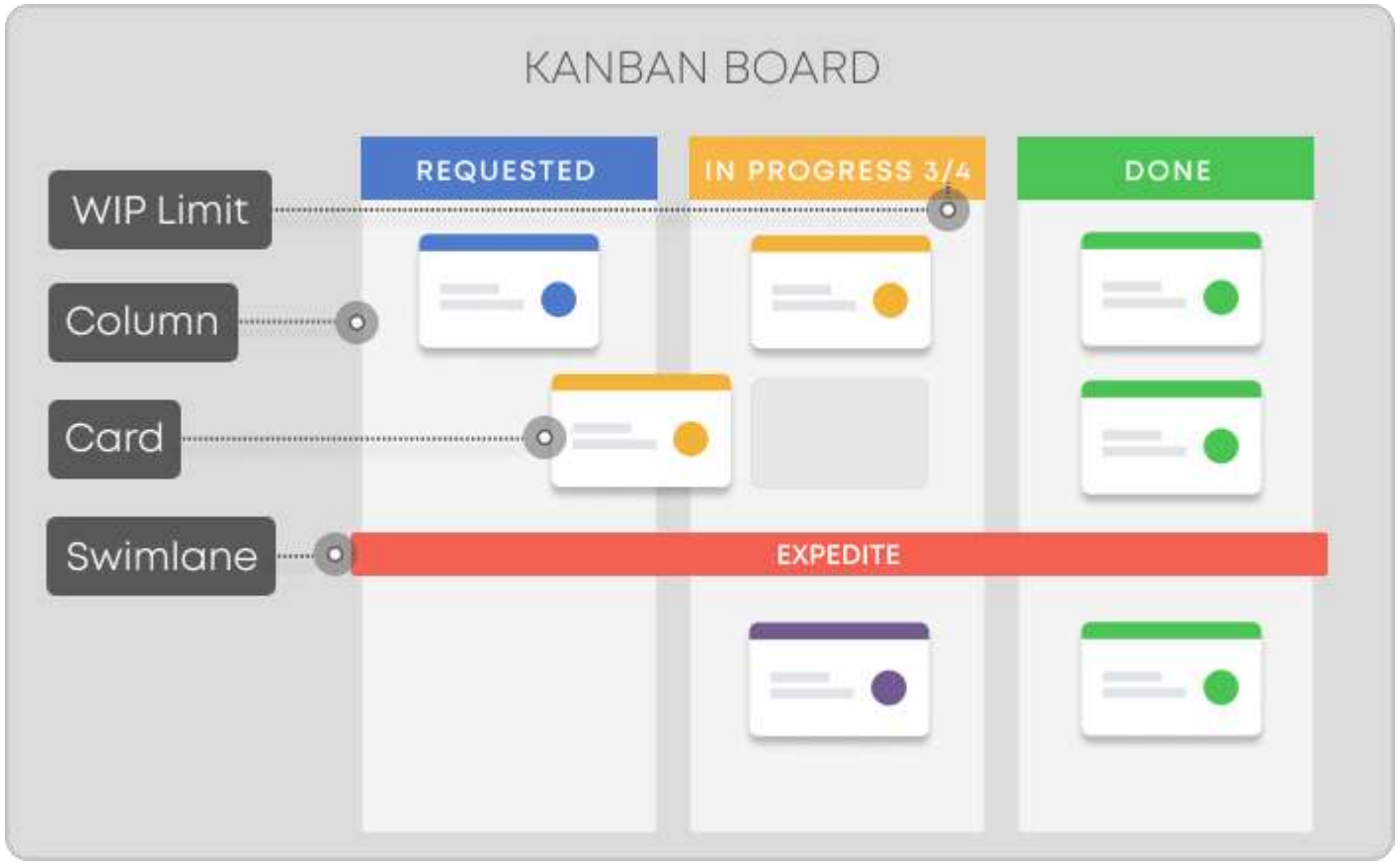
This is the definition of “Done” for the Scrum Team and is used to assess when work is complete on the product Increment

看板 Kanban


KANBAN BOARD
REQUESTED
IN PROGRESS 3/4
DONE
WIP Limit
Column
Card
Swimlane
EXPEDITE

What is KANBAN?

Kanban is a popular framework used to implement agile and DevOps software development. It requires real-time communication of capacity and full transparency of work. Work items are represented visually on a kanban board, allowing team members to see the state of every piece of work at any time.



The Japanese word “kanban”, meaning “visual board” or a “sign”, has been used in the sense of a process definition since the 1950s. It was first developed and applied by Toyota as a scheduling system for just-in-time manufacturing. On the other hand, the capitalized term “Kanban” is known and associated with the emergence of the “Kanban Method,” which was first defined in 2007.

Kanban is a method to gradually improve whatever you do, whether software development, IT/ Ops, Staffing, Recruitment, Marketing and Sales or Procurement. In fact, almost any business function can benefit from applying Kanban to bring about significant benefits such as reduced lead time, increased throughput and much higher quality of products or services delivered.

https://kanbanize.com/kanban-resources/getting-started/what-is-kanban

https://www.tutorialspoint.com/kanban/index.htm

The core concept of Kanban

Visualize Workflow

- Split the entire work into defined segments or states, visualized as named columns on a wall.
- Write each item on a card and put in a column to indicate where the item is in the workflow.

Limit WIP

- Assign explicit limits to how many items can be in progress at each workflow segment / state. i.e., Work in Progress (WIP) is limited in each workflow state.

Measure the Lead Time

- Lead Time, also known as cycle time is the average time to complete one item. Measure the Lead Time and optimize the process to make the Lead Time as small and predictable as possible.

Kanban - Benefits

Bottlenecks become clearly visible in real-time. This leads people to collaborate to optimize the whole value chain rather than just their part.

Useful for situations where operations and support teams have a high rate of uncertainty and variability.

Tends to spread throughout the organization naturally, including sales and management. This increases visibility of everything that is going on at the company.

Reduces inventory in the range of 25%-75%, thereby reducing company costs.

Since all segments/states in the workflow are visually organized, the required items, reducing the wait times and ensuring speed, continually support all the tasks in the workflow.

Overproduction of inventory is avoided, thereby saving resources and time as well. This is termed as eliminating waste.

Alignment with Agile

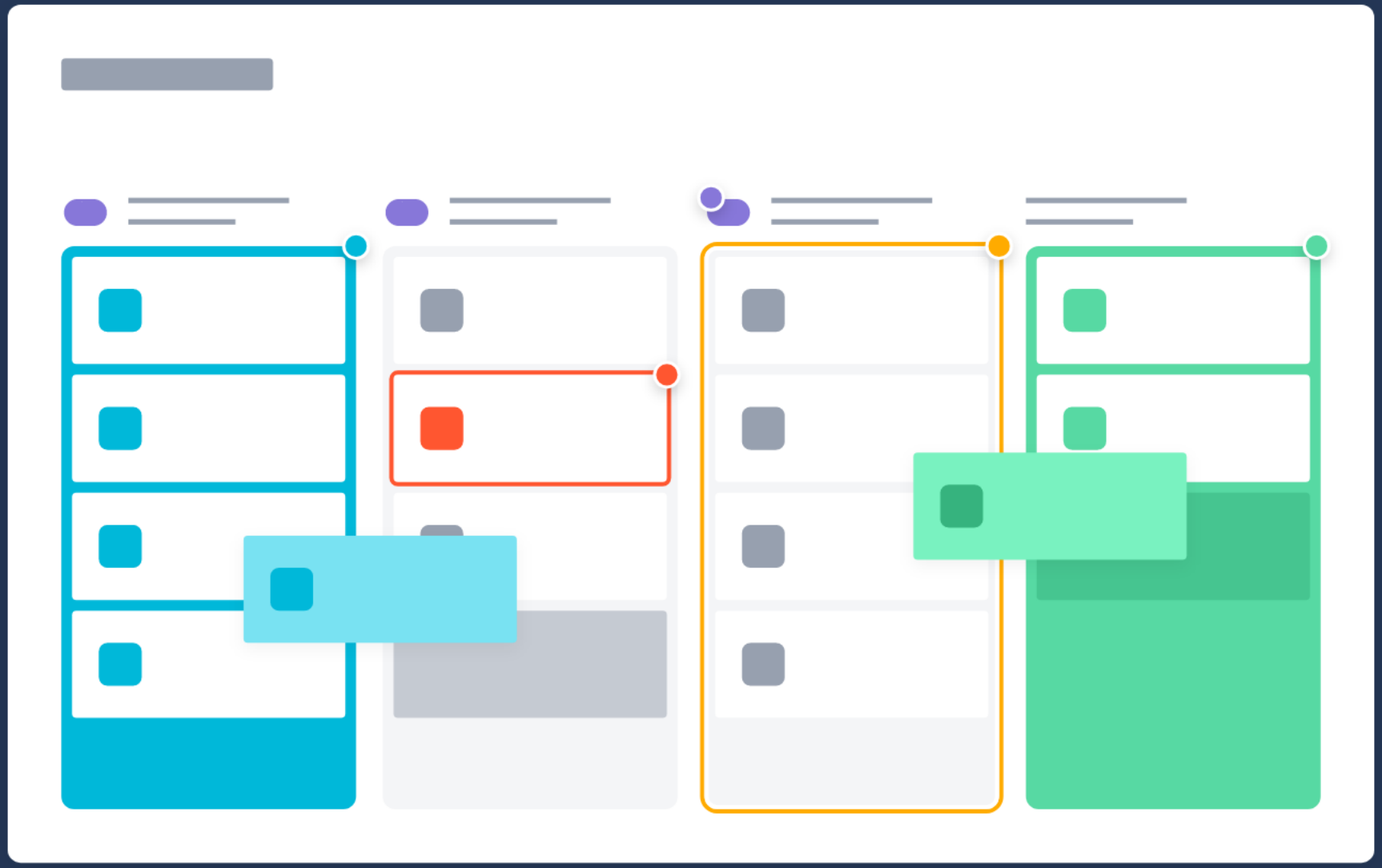
In agile, if values are combined with Kanban characteristics, the outcome would be Agile Kanban. This practice is gaining popularity in Software Development wherein the Agile iteration approach and Kanban value stream focus are combined.

What is a kanban board?

A kanban board is an agile project management tool designed to help visualize work, limit work-in-progress, and maximize efficiency (or flow). It can help both agile and DevOps teams establish order in their daily work. Kanban boards use cards, columns, and continuous improvement to help technology and service teams commit to the right amount of work, and get it done!

Kanban Board

Visual signal
Columns
Work-in-progress limit
Commitment point
Delivery point



Visual Signals — One of the first things you'll notice about a kanban board are the visual cards (stickies, tickets, or otherwise). Kanban teams write all of their projects and work items onto cards, usually one per card. For agile teams, each card could encapsulate one user story. Once on the board, these visual signals help teammates and stakeholders quickly understand what the team is working on.

Columns — Another hallmark of the kanban board are the columns. Each column represents a specific activity that together compose a "workflow". Cards flow through the workflow until completion. Workflows can be as simple as "To Do," "In Progress," "Complete," or much more complex.

Work In Progress (WIP) Limits — WIP limits are the maximum number of cards that can be in one column at any given time. A column with a WIP limit of three cannot have more than three cards in it. When the column is "maxed-out" the team needs to swarm on those cards and move them forward before new cards can move into that stage of the workflow. These WIP limits are critical for exposing bottlenecks in the workflow and maximizing flow. WIP limits give you an early warning sign that you committed to too much work.

Commitment point — Kanban teams often have a backlog for their board. This is where customers and teammates put ideas for projects that the team can pick up when they are ready. The commitment point is the moment when an idea is picked up by the team and work starts on the project.

Delivery point — The delivery point is the end of a kanban team's workflow. For most teams, the delivery point is when the product or service is in the hands of the customer. The team's goal is to take cards from the commitment point to the delivery point as fast as possible. The elapsed time between the two is the called Lead Time. Kanban teams are continuously improving to decrease their lead time as much as possible.

What are WIP limits?

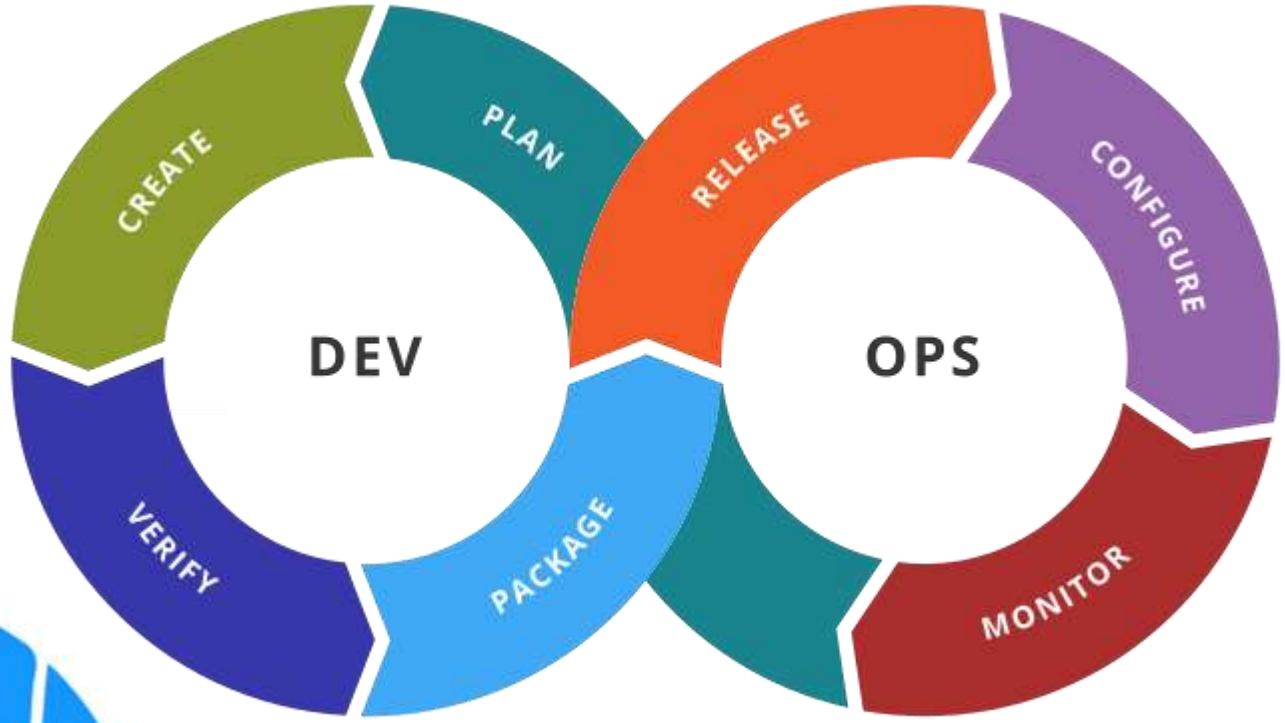
In agile development, work in progress (WIP) limits set the maximum amount of work that can exist in each status of a workflow. Limiting the amount of work in progress makes it easier to identify inefficiency in a team's workflow. Bottlenecks in a team's delivery pipeline are clearly visible before a situation becomes dire.

What Is DevOps *(development & operations)*?

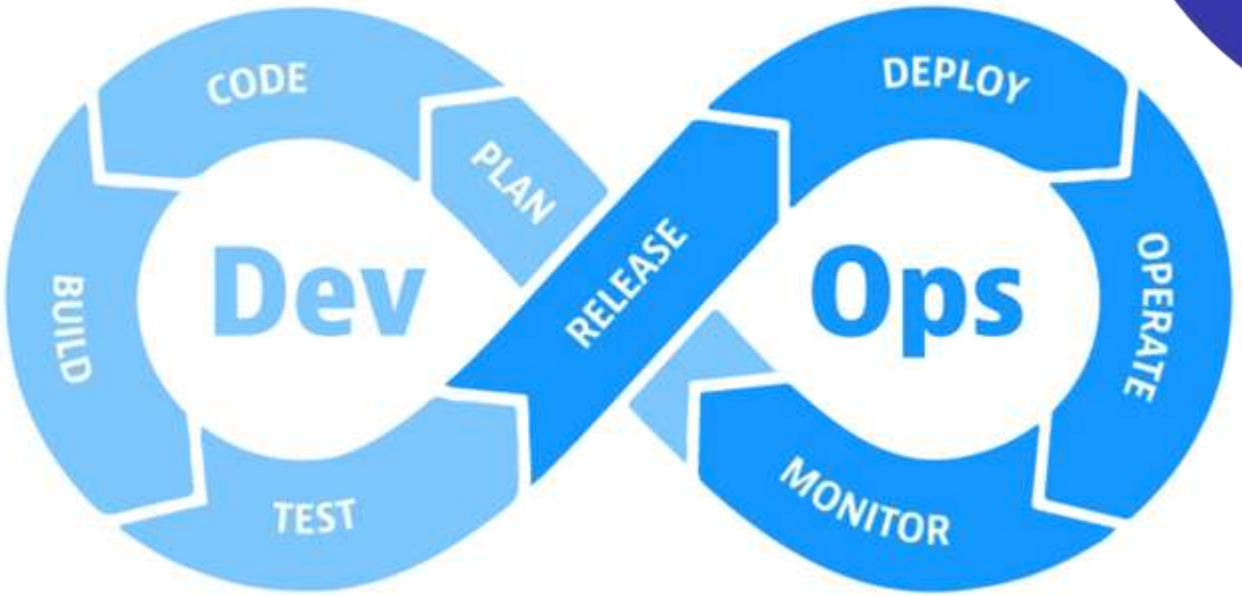
DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.

A DevOps team includes developers and IT operations working collaboratively throughout the product lifecycle, in order to increase the speed and quality of software deployment. It's a new way of working, a cultural shift, that has significant implications for teams and the organizations they work for.

The DevOps lifecycle



KOENIG
step forward



Plan

DevOps teams should adopt agile practices to improve speed and quality. Agile is an iterative approach to project management and software development that helps teams break work into smaller pieces to deliver incremental value.

Operate

Manage the end-to-end delivery of IT services to customers. This includes the practices involved in design, implementation, configuration, deployment, and maintenance of all IT infrastructure that supports an organization’s services.

5 Key DevOps principles

To realize the full potential of DevOps, teams should follow key DevOps principles

Collaboration

The key premise behind DevOps is collaboration. Development and operations teams coalesce into a functional team that communicates, shares feedback, and collaborates throughout the entire development and deployment cycle. Often, this means development and operations teams merge into a single team that works across the entire application lifecycle.

The members of a DevOps team are responsible for ensuring quality deliverables across each facet of the product. This leads to more ‘full stack’ development, where teams own the complete backend-to-frontend responsibilities of a feature or product. Teams will own a feature or project throughout the complete lifecycle from idea to delivery. This enhanced level of investment and attachment from the team leads to higher quality output.

Automation

An essential practice of DevOps is to automate as much of the software development lifecycle as possible. This gives developers more time to write code and develop new features. Automation is a key element of a CI/CD pipeline and helps to reduce human errors and increase team productivity. With automated processes, teams achieve continuous improvement with short iteration times, which allows them to quickly respond to customer feedback.

Continuous Improvement

Continuous improvement was established as a staple of agile practices, as well as lean manufacturing and Improvement Kata. It's the practice of focusing on experimentation, minimizing waste, and optimizing for speed, cost, and ease of delivery. Continuous improvement is also tied to continuous delivery, allowing DevOps teams to continuously push updates that improve the efficiency of software systems. The constant pipeline of new releases means teams consistently push code changes that eliminate waste, improve development efficiency, and bring more customer value.

Customer-centric action

DevOps teams use short feedback loops with customers and end users to develop products and services centered around user needs. DevOps practices enable rapid collection and response to user feedback through use of real-time live monitoring and rapid deployment. Teams get immediate visibility into how live users interact with a software system and use that insight to develop further improvements.

Create with the end in mind

This principle involves understanding the needs of customers and creating products or services that solve real problems. Teams shouldn’t ‘build in a bubble’, or create software based on assumptions about how consumers will use the software. Rather, DevOps teams should have a holistic understanding of the product, from creation to implementation.

Benefits of DevOps

There are three primary categories of benefits for DevOps:

- technical benefits
- cultural benefits
- business benefits.

The technical benefits include reduced complexity, continuous delivery, and faster problem resolution. DevOps practitioners typically deploy higher-quality code faster than traditional siloed teams.

The cultural benefits are more productive and efficient teams, and happier customers.

On the business end, the benefits include greater collaboration and trust between team members, which results in faster delivery and stable operating environments.